The Chinese University of Hong Kong, Shenzhen
School of Data Science



DDA5001 · Homework 1
Due: 23:59, Sep 30.

Instructions:

- Homework problems must be clearly answered to receive full credit. Complete sentences that establish a clear logical progress are highly recommended.
- Submit your answer paper in Blackboard. Prepare your answers in a **PDF** file and saving all your code as **.py** files. Combine the PDF and code files into a single .zip file for submission. The file name should be in the format **last name-first name-hw1**; e.g., **San-Zhang-hw1**.
- You may submit your homework multiple times before the deadline. Only your **latest submission will be graded**.
- The homework must be written in English.
- Late submission will not be graded.
- Each student **must not copy** homework solutions from another student or from any other source. You are encouraged to discuss with others. However, you must write down the answer using your understanding (after discussion) and words.

Problem 1 (12 points). Concepts and Fundamental Knowledge

(a) (4 points) Concisely state the difference between supervised learning and unsupervised learning.

(b) (4 points) Which one of the following statements is true?

1) Regression is used to fit categorical labels.
2) Least squares must have infinitely many solutions if the number of data points is smaller than feature dimension.
3) The perceptron always converges to a unique linear classifier for a given training dataset.
4) Least squares is a maximum likelihood estimator regardless of the noise distribution.

(c) (4 points) Let the data matrix $\boldsymbol{X} \in \mathbb{R}^{n\times d}$ be a full column rank matrix. Explain why $\boldsymbol{X}^\top \boldsymbol{X}$ is positive definite (PD). (Hint: Use the definition of linear independence)

Since a PD matrix must be invertible as well, this clarifies why we can take matrix inverse to obtain the unique solution of least squares in case I (where $\boldsymbol{X}$ has full column rank). From an optimization perspective, this PD property also reveals that the least squares in case I is strongly convex.

Solution.
(a) The supervised learning require the label of each data point to learn, while the unsupervised learning does not.

(b) 2). For 1), regression is for continuous labels. For 3), perceptron may have infinitely many solutions, any line that can classify the two classes. For 4), least squares is a maximum likelihood estimator in a linear model with i.i.d. additive noise $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$, $\sigma^2 > 0$, but not regardless of the noise distribution.

(c) For any vector $\boldsymbol{v} \in \mathbb{R}^d \setminus \{0\}$, we have

$$\boldsymbol{v}^\top (\boldsymbol{X}^\top \boldsymbol{X}) \boldsymbol{v} = (\boldsymbol{X}\boldsymbol{v})^\top (\boldsymbol{X}\boldsymbol{v}) = \|\boldsymbol{X}\boldsymbol{v}\|^2 \geq 0.$$

$\|\boldsymbol{X}\boldsymbol{v}\|^2 = 0$ if and only if $\boldsymbol{X}\boldsymbol{v} = 0$ (recall $\boldsymbol{v} \neq 0$). However, this is not possible since $\boldsymbol{X}$ has linear independent columns due to the fact that is has full column rank. Thus, $\boldsymbol{v}^\top (\boldsymbol{X}^\top \boldsymbol{X}) \boldsymbol{v} > 0$ for any vector $\boldsymbol{v} \in \mathbb{R}^d \setminus \{0\}$, showing that $\boldsymbol{X}^\top \boldsymbol{X}$ is PD.

■

Problem 2 (13 points). Least Squares Without Full Column Rank

Consider the least squares problem

$$\min_{\boldsymbol{\theta}\in\mathbb{R}^d} \|\boldsymbol{X}\boldsymbol{\theta}-\boldsymbol{y}\|_2^2,$$

where $\boldsymbol{X}\in\mathbb{R}^{n\times d}$, $\boldsymbol{\theta}\in\mathbb{R}^d$, $\boldsymbol{y}\in\mathbb{R}^n$.

(a) (8 points) When $n<d$, the problem yields infinite many solutions. Assume that $\text{rank}(\boldsymbol{X})=n$, derive the expression of the infinitely many solutions based on singular value decomposition (SVD). In addition, what is the optimal function value?

Note that the SVD is a notion from linear algebra, and the SVD of the matrix $\boldsymbol{X}$ can be written as

$$\boldsymbol{X}=\underbrace{\boldsymbol{V}}_{\in\mathbb{R}^{n\times n}}\underbrace{\begin{bmatrix}\boldsymbol{\Sigma}_1 & \boldsymbol{0}\end{bmatrix}}_{\in\mathbb{R}^{n\times d}}\underbrace{\begin{bmatrix}\boldsymbol{U}_1^\top\\ \boldsymbol{U}_2^\top\end{bmatrix}}_{\in\mathbb{R}^{d\times d}}=\boldsymbol{V}\boldsymbol{\Sigma}_1\boldsymbol{U}_1^\top.$$

where $\boldsymbol{V}\in\mathbb{R}^{n\times n}$ is an orthogonal matrix such that $\boldsymbol{V}^\top\boldsymbol{V}=\boldsymbol{V}\boldsymbol{V}^\top=\mathbf{I}$, $\boldsymbol{\Sigma}_1\in\mathbb{R}^{n\times n}$ is a diagonal matrix taking the form

$$\boldsymbol{\Sigma}_1(i,j)=\begin{cases}\sigma_i, & i=j\\ 0, & i\neq j\end{cases}\quad\text{with}\quad\sigma_1\geq\sigma_2\geq\cdots\geq\sigma_n>0,$$

$\boldsymbol{U}_1^\top\in\mathbb{R}^{n\times d}$ is an semi-orthogonal matrix satisfying $\boldsymbol{U}_1^\top\boldsymbol{U}_1=\mathbf{I}$ (but $\boldsymbol{U}_1\boldsymbol{U}_1^\top\neq\mathbf{I}$), and $\boldsymbol{U}_2^\top\in\mathbb{R}^{(d-n)\times d}$ is an semi-orthogonal matrix satisfying $\boldsymbol{U}_2^\top\boldsymbol{U}_2=\mathbf{I}$ (but $\boldsymbol{U}_2\boldsymbol{U}_2^\top\neq\mathbf{I}$).

(Hint: Let $\boldsymbol{A}:=\boldsymbol{V}\boldsymbol{\Sigma}_1$ which is square matrix of full rank, $\boldsymbol{z}=\boldsymbol{U}_1^\top\boldsymbol{\theta}$, solve $\min_{\boldsymbol{z}}\|\boldsymbol{A}\boldsymbol{z}-\boldsymbol{y}\|_2^2$ first, then solve $\boldsymbol{U}_1^\top\boldsymbol{\theta}=\boldsymbol{z}$ by considering that $\boldsymbol{U}_1^\top\boldsymbol{U}_2=\mathbf{0}$.)

(b) (5 points) Later in the overfitting section, we will study that we can also add a regularizer to solve the issue of infinitely many solutions, resulting in the following problem:

$$\min_{\boldsymbol{\theta}\in\mathbb{R}^d} \|\boldsymbol{X}\boldsymbol{\theta}-\boldsymbol{y}\|_2^2+\lambda\|\boldsymbol{\theta}\|_2^2,$$

where λ is a positive constant. Derive the unique solution of this problem.

Solution.

(a) By the close form solution of full column rank least square, we have

$$\begin{aligned}\boldsymbol{z}&=\left((\boldsymbol{A}^\top\boldsymbol{A})^{-1}\boldsymbol{A}^\top\boldsymbol{y}\right)\\&=\left(\boldsymbol{\Sigma}_1^\top\boldsymbol{V}^\top\boldsymbol{V}\boldsymbol{\Sigma}_1\right)^{-1}\boldsymbol{\Sigma}_1^\top\boldsymbol{V}^\top\boldsymbol{y}\\&=\boldsymbol{\Sigma}_1^{-1}\boldsymbol{V}^\top\boldsymbol{y}.\end{aligned}$$

Hence, the optimality condition of $\boldsymbol{\theta}$ is given by

$$\begin{aligned}
&\boldsymbol{U}_1^\top \boldsymbol{\theta} = \boldsymbol{\Sigma}_1^{-1} \boldsymbol{V}^\top \boldsymbol{y} \\
\iff &\boldsymbol{\theta} = \boldsymbol{U}_1 \boldsymbol{\Sigma}_1^{-1} \boldsymbol{V}^\top \boldsymbol{y} + \boldsymbol{U}_2 \boldsymbol{s},\ \forall \boldsymbol{s} \in \mathbb{R}^{d-n} \\
\iff &\boldsymbol{\theta} = \boldsymbol{U}_1 \boldsymbol{\Sigma}_1^{-1} \boldsymbol{V}^\top \boldsymbol{y} + \boldsymbol{h},\ \forall \boldsymbol{h} \in \mathrm{null}(\boldsymbol{U}_1^\top) = \mathrm{null}(\boldsymbol{X}).
\end{aligned}$$

The optimal function value is 0 by plugging $\boldsymbol{\theta}$ back to the objective function.

(b) The gradient of the objective is given by

$$2\boldsymbol{X}^\top (\boldsymbol{X}\boldsymbol{\theta} - \boldsymbol{y}) + 2\lambda \mathbf{I} \boldsymbol{\theta}.$$

Setting the gradient to $\mathbf{0}$ gives

$$(\boldsymbol{X}^\top \boldsymbol{X} + \lambda \mathbf{I})\boldsymbol{\theta} = \boldsymbol{X}^\top \boldsymbol{y}.$$

Since for any vector $\boldsymbol{z} \in \mathbb{R}^d \setminus \mathbf{0}$, we have $\boldsymbol{z}^\top \left(\boldsymbol{X}^\top \boldsymbol{X} + \lambda \mathbf{I}\right) \boldsymbol{z} > 0$, the matrix $\boldsymbol{X}^\top \boldsymbol{X} + \lambda \mathbf{I}$ is positive definite and thereby invertible, we have

$$\boldsymbol{\theta} = \left(\boldsymbol{X}^\top \boldsymbol{X} + \lambda \mathbf{I}\right)^{-1} \boldsymbol{X}^\top \boldsymbol{y}.$$

■

Problem 3 (23 points). Robust Linear Regression with a Coding Agent

AI collaboration for Problems 3 and 5. For these two problems, you are encouraged to use a coding agent to help with derivation, implementation, debugging, and analysis. This permission takes precedence over the general restriction on using other sources insofar as it concerns AI assistance on these two problems. You remain responsible for understanding and explaining your submission. Copying another student's work or fabricating conversations or experimental results is not permitted.

For each problem, record one initial, testable expectation before running your verification, use an agent to help implement the task, and bring a concrete verification result back to the agent for a focused follow-up. Explain your final decision using mathematical or numerical evidence. An initially correct agent solution can receive full credit. There is no requirement to find an AI error or to produce a minimum number of chat turns.

Use `code_source/p3/p3.py`. Data loading, plotting, and output scaffolding are provided. Complete the marked mathematical and verification functions, with agent assistance. Submit your completed code and numerical outputs together with your answer PDF. Include one AI collaboration case for this problem, at most one page, as specified in part (e).

Suppose the linear regression model is

$$\boldsymbol{y} = \boldsymbol{X}\boldsymbol{\theta}^\star + \boldsymbol{\epsilon}, \qquad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \sigma^2 I), \quad \sigma > 0,$$

where $\boldsymbol{X} \in \mathbb{R}^{n \times d}$ has full column rank. Under this noise assumption, the maximum likelihood estimator is the least-squares solution

$$\widehat{\boldsymbol{\theta}}_{LS} = \operatorname*{argmin}_{\boldsymbol{\theta}} \|\boldsymbol{X}\boldsymbol{\theta} - \boldsymbol{y}\|_2^2 = (\boldsymbol{X}^\top \boldsymbol{X})^{-1} \boldsymbol{X}^\top \boldsymbol{y}.$$

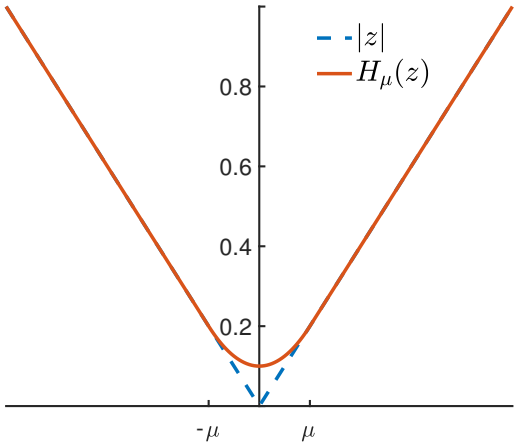
(a) **(3 points) Laplace noise and the loss function.** Instead suppose that the errors are i.i.d. Laplace with density

$$p(\varepsilon_i) = \frac{1}{2b} \exp(-|\varepsilon_i|/b), \qquad b > 0.$$

Derive the optimization problem for estimating $\boldsymbol{\theta}^\star$ by maximum likelihood. Show the negative log-likelihood and identify the terms that do not depend on $\boldsymbol{\theta}$.

(b) **(3 points) Huber smoothing and its gradient.** For $\mu > 0$, define the following smoothing of the absolute value:

$$h_\mu(z) = \begin{cases} |z|, & |z| \geq \mu, \\ \dfrac{z^2}{2\mu} + \dfrac{\mu}{2}, & |z| < \mu, \end{cases} \qquad \mathcal{L}(\boldsymbol{\theta}) = \sum_{i=1}^{n} h_\mu((\boldsymbol{X}\boldsymbol{\theta} - \boldsymbol{y})_i).$$



Geometric intuition. The blue dashed curve is $|z|$, and the orange curve is its smoothing $h_\mu(z)$. On $(-\mu, \mu)$, a quadratic arc replaces the sharp corner of $|z|$; outside this interval, the two curves coincide. This makes the loss continuously differentiable for gradient-based optimization, while retaining linear growth for large residuals, so outliers have less influence than under squared loss.

Derive $h'_\mu(z)$ and $\nabla\mathcal{L}(\boldsymbol{\theta})$, with the dimensions of the vectors in your expression. Use this exact definition of h_μ and a *sum* over samples in the implementation. A library function named "Huber" may use a different scaling or additive constant.

(c) **(6 points) Agent-assisted implementation and baseline experiment.** The supplied data follow

$$\boldsymbol{y} = \boldsymbol{X}\boldsymbol{\theta}^\star + \boldsymbol{\epsilon}_1 + \boldsymbol{\epsilon}_2,$$

where $\boldsymbol{\epsilon}_1$ is Gaussian noise and $\boldsymbol{\epsilon}_2$ contains sparse outliers. Load the supplied X, y, and $\theta^\star$ from `data/`; $n = 1000$ and $d = 50$.

(1) Compute the least-squares estimate and report $\|\widehat{\boldsymbol{\theta}}_{LS} - \boldsymbol{\theta}^\star\|_2$. Use a numerical least-squares solver rather than explicitly forming a matrix inverse.

(2) Implement

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \alpha\nabla\mathcal{L}(\boldsymbol{\theta}_k), \qquad k = 0, \ldots, T-1,$$

using $\mu = 10^{-5}$, $\alpha = 0.001$, and $T = 1000$. Use the provided initialization:

```
rng = np.random.default_rng(0)
theta0 = rng.standard_normal(d)
```

Record $T + 1$ states, including $k = 0$ and $k = T$.

(3) Plot $\|\boldsymbol{\theta}_k - \boldsymbol{\theta}^\star\|_2$ versus k, with the least-squares error as a horizontal reference. Save the objective values and parameter errors as a CSV file. Report the seed and command needed to reproduce the outputs. Use $\boldsymbol{\theta}^\star$ only for evaluation, never to compute an update or choose an iterate.

(d) **(5 points) Verify the agent's implementation.** Complete the verification TODOs and submit a small table of numerical results.

(1) Check the scalar derivative at residuals $(-2, -0.5, 0, 0.5, 2)\mu$, covering both outer branches and the inner branch. Then use the supplied small matrix to compare a directional derivative from your gradient with a finite-difference approximation at several step sizes. Run this check with $\mu = 1$ and with $\mu = 10^{-5}$. Explain how it is related to the first-order Taylor expansion from slides 1, and why an excessively small or large finite-difference step can be misleading. A table of "passed" labels without the compared values and an explanation is insufficient.

(2) Verify on the same small example that replacing the sum loss by the mean loss divides the gradient by the number of samples. Explain how the learning rate would have to change to obtain the same update. Check whether the agent's baseline code implements the sum or the mean.

The small example and step-size scaffolding are supplied so that this check requires no additional optimization theory.

(e) **(6 points) AI collaboration case, at most one page.** Document one interaction connected to parts (b)–(d). Include: (i) your initial expectation and the issue you wanted to check; (ii) the tool/model, your actual prompt, and a short relevant excerpt of its response; (iii) the numerical evidence you brought back to the agent and a relevant excerpt of the follow-up; and (iv) your decision to revise or retain the result, supported by the derivative formula, verification table, or code. Reference your submitted files rather than duplicating all outputs. Retain the relevant original conversation for possible review. Do not manufacture a failure if the first implementation was correct.

Problem 3: reference solution and marking guide

(a) The negative log-likelihood is

$$-\log p(\boldsymbol{y} \mid \boldsymbol{X}, \boldsymbol{\theta}) = n \log(2b) + \frac{1}{b} \sum_{i=1}^{n} |y_i - \boldsymbol{x}_i^\top \boldsymbol{\theta}|.$$

For fixed $b > 0$, maximizing likelihood therefore gives $\min_{\boldsymbol{\theta}} \|\boldsymbol{X}\boldsymbol{\theta} - \boldsymbol{y}\|_1$. *Marking:* likelihood/log transformation, 2 points; objective and explanation of constants, 1 point.

(b) Writing $r = \boldsymbol{X}\boldsymbol{\theta} - \boldsymbol{y} \in \mathbb{R}^n$, define

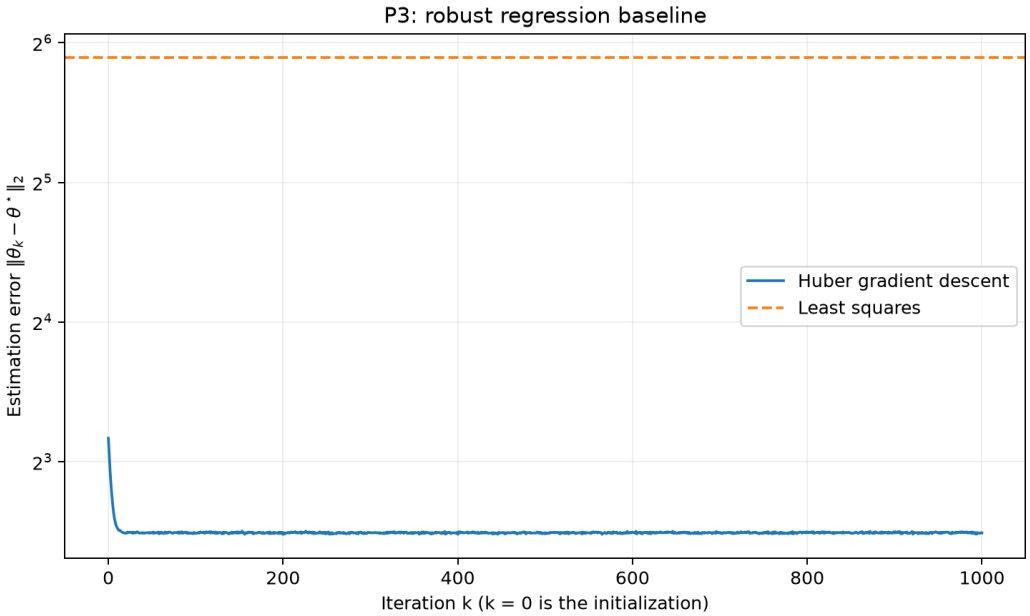
$$\psi_\mu(z) = h'_\mu(z) = \begin{cases} -1, & z \le -\mu, \\ z/\mu, & -\mu < z < \mu, \\ 1, & z \ge \mu. \end{cases}$$

The values agree at $\pm\mu$. The gradient is

$$\nabla \mathcal{L}(\boldsymbol{\theta}) = \boldsymbol{X}^\top \begin{bmatrix} \psi_\mu(r_1) & \cdots & \psi_\mu(r_n) \end{bmatrix}^\top \in \mathbb{R}^d.$$

The loss in the question equals a commonly used unscaled Huber loss divided by μ, plus $\mu/2$ per sample. The constant changes reported objective values but not gradients. A multiplicative factor changes both gradients and fixed-step updates. *Marking:* scalar branches, 1 point; chain rule and dimensions, 2 points.

(c) The full reference implementation is `code_solution/p3/p3.py`. It uses `np.linalg.lstsq`, flattens y and $\theta^\star$ to prevent accidental broadcasting, and performs exactly 1000 gradient updates. With seed 0, the least-squares parameter error is 59.513636, and the Huber parameter error changes from 8.989940 at initialization to 5.618720 after 1000 updates. The supplied script saves full-precision values in `summary.json` and the complete trajectory as a CSV file. The figure below is generated by that implementation on the supplied data.



The step size and smoothing parameter define the requested baseline; they do not imply that the parameter error decreases at every step. In particular, a decrease in the objective need not imply a decrease in distance to $\theta^\star$. *Marking:* least-squares computation and error, 2 points; correct baseline updates and curve, 3 points; reproducible initialization, history, and command, 1 point.

(d) At the five specified residuals, the scalar derivatives are $(-1, -0.5, 0, 0.5, 1)$. For a direction v, compare

$$D_h = \frac{\mathcal{L}(\theta + hv) - \mathcal{L}(\theta - hv)}{2h} \quad \text{with} \quad \nabla\mathcal{L}(\theta)^\top v.$$

First-order differentiability predicts agreement as h becomes small in exact arithmetic. A step that crosses a significant part of the smoothing region may have substantial truncation error, while extremely small steps can suffer cancellation. The supplied checks scale steps with μ, include inner and outer residuals, and report the quantities being compared. Merely checking the shape of the gradient would not detect an incorrect sign or scaling.

For the supplied five-sample check, $\theta = 0$ and $v = (3, 4)^\top/5$ give $\nabla\mathcal{L} = (0.5, 1)^\top$ and $\nabla\mathcal{L}^\top v = 1.1$, for both tested values of μ. At $h = 10^{-4}\mu$, the directional finite difference agrees within 3×10^{-12} in the reference run. The mean-loss directional derivative is 0.22. Inspect the full table rather than expecting finite-precision errors to decrease at every step size.

If $\overline{\mathcal{L}} = \mathcal{L}/n$, then $\nabla\overline{\mathcal{L}} = \nabla\mathcal{L}/n$. Matching a sum-loss update with learning rate α requires a mean-loss learning rate $n\alpha$. This comparison is about the update, not about a change in the minimizer. *Marking:* meaningful branch/directional checks with values and explanation, 3 points; sum/mean numerical comparison and learning-rate reasoning, 2 points.

(e) This part has no canned conversation answer. Award 1 point for a specific expectation/checking question, 2 points for relevant genuine interaction evidence including an evidence-based follow-up, and 3 points for a mathematically supported final decision linked to the submitted code or numerical outputs. Correct code on the first attempt is fully acceptable. Do not reward extra chat turns, excessive screenshots, or an invented debugging narrative.

Problem 4 (24 points). Convergence of The Perceptron for Linearly Separable Data

In lecture, we studied that one iteration of perceptron algorithm will move forward to the currently wrongly classified data. Though such an observation does provide some intuition that perceptron will eventually converge to a linear classifier that correctly separate all the (linearly separable) training data, it is not rigorously justified.

Let us first review the algorithm. The perceptron applies to the linear model

$$f_{\boldsymbol{\theta}}(\boldsymbol{x}) = \boldsymbol{\theta}^\top \boldsymbol{x}.$$

Here $\boldsymbol{x}$ includes a constant 1 as a bias coordinate, and $y_i \in \{+1, -1\}$. The diagrams show the original two-dimensional feature space. Define $\text{sign}(z) = +1$ for $z \geq 0$ and $\text{sign}(z) = -1$ otherwise, as in Problem 5.

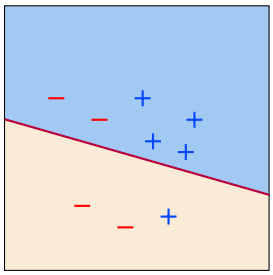
At the k-th mistake update, the current parameter is $\boldsymbol{\theta}_{k-1}$. Choose any misclassified training sample, denoted by $(\boldsymbol{x}_{k-1}, y_{k-1})$ for this update, so that

$$\text{sign}\left(\boldsymbol{\theta}_{k-1}^\top \boldsymbol{x}_{k-1}\right) \neq y_{k-1}.$$

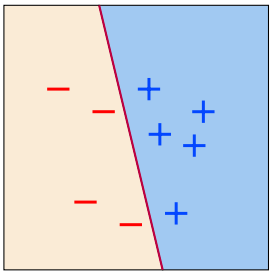
Then, perceptron updates as

$$\boldsymbol{\theta}_k = \boldsymbol{\theta}_{k-1} + y_{k-1}\boldsymbol{x}_{k-1} \quad \forall k = 1, 2, \ldots \tag{1}$$

In this question, you will be guided to show that perceptron terminates after finitely many mistake updates on strictly linearly separable data; see Fig. 1. The count k refers to parameter updates, not full passes through the dataset (epochs).



(a) Perceptron at the k-th iteration, not all data are correctly classified



(b) Learned perceptron model after at most $\bar{k}$ iterations, all data are correctly classified

Figure 1: Illustration of the convergence progress of the perceptron learning algorithm.

Assume the data are strictly linearly separable: choose $\boldsymbol{\theta}^*$ such that $y_i \boldsymbol{\theta}^{*\top} \boldsymbol{x}_i > 0$ for every training sample i, as illustrated in Fig. 1b. For simplicity, initialize the perceptron at $\boldsymbol{\theta}_0 = \mathbf{0}$.

The full proof procedure is divided into 5 steps.

(a) (3 points) Let $\rho = \min_{1 \leq i \leq n} y_i(\boldsymbol{\theta}^{*\top} \boldsymbol{x}_i)$. Show that $\rho > 0$.

Solution: Every $y_i \boldsymbol{\theta}^{*\top} \boldsymbol{x}_i$ is strictly positive by the separation assumption. The minimum over the finite training set is therefore strictly positive.

(b) (6 points) Show that $\boldsymbol{\theta}_k^\top \boldsymbol{\theta}^* \geq \boldsymbol{\theta}_{k-1}^\top \boldsymbol{\theta}^* + \rho$, and hence conclude that $\boldsymbol{\theta}_k^\top \boldsymbol{\theta}^* \geq k\rho$.

(Hint: Use the update (1).)

Solution: From the update (1), we have $\boldsymbol{\theta}_k^\top \boldsymbol{\theta}^* = (\boldsymbol{\theta}_{k-1} + y_{k-1}\boldsymbol{x}_{k-1})^\top \boldsymbol{\theta}^* = \boldsymbol{\theta}_{k-1}^\top \boldsymbol{\theta}^* + y_{k-1}\boldsymbol{x}_{k-1}^\top \boldsymbol{\theta}^* \geq \boldsymbol{\theta}_{k-1}^\top \boldsymbol{\theta}^* + \rho$, where the last inequality is from the definition of ρ. Now, we just unroll this inequality from k-th iteration to 0-th iteration and use $\boldsymbol{\theta}_0 = \mathbf{0}$, we have $\boldsymbol{\theta}_k^\top \boldsymbol{\theta}^* \geq k\rho$.

(c) (6 points) Show that $\|\boldsymbol{\theta}_k\|^2 \leq \|\boldsymbol{\theta}_{k-1}\|^2 + \|\boldsymbol{x}_{k-1}\|^2$.

(Hint: Use the fact that $\boldsymbol{x}_{k-1}$ is misclassified by $\boldsymbol{\theta}_{k-1}$.)

Solution: From the update (1),

$$\begin{aligned}\|\boldsymbol{\theta}_k\|^2 &= \|\boldsymbol{\theta}_{k-1} + y_{k-1}\boldsymbol{x}_{k-1}\|^2 \\ &= \|\boldsymbol{\theta}_{k-1}\|^2 + 2y_{k-1}\boldsymbol{\theta}_{k-1}^\top \boldsymbol{x}_{k-1} + \|\boldsymbol{x}_{k-1}\|^2,\end{aligned}$$

where $y_{k-1}^2 = 1$. Misclassification under the stated zero-score rule implies $y_{k-1}\boldsymbol{\theta}_{k-1}^\top \boldsymbol{x}_{k-1} \leq 0$, which gives the desired inequality. Equality can occur when the score is zero, including at initialization.

(d) (3 points) Show that $\|\boldsymbol{\theta}_k\|^2 \leq kR^2$, where $R = \max_{1 \leq i \leq n} \|\boldsymbol{x}_i\|$.

Solution: By unrolling the inequality in the last step from k-th iteration to 0-th iteration and $\boldsymbol{\theta}_0 = \mathbf{0}$, we obtain the desired result.

(e) (6 points) For $k \geq 1$ after k mistake updates, use steps 2 and 4 to show that

$$\frac{\boldsymbol{\theta}_k^\top \boldsymbol{\theta}^*}{\|\boldsymbol{\theta}_k\|} \geq \sqrt{k}\frac{\rho}{R}$$

and hence show that the perceptron learning algorithm must stop within at most

$$\bar{k} \leq \frac{R^2\|\boldsymbol{\theta}^*\|^2}{\rho^2}$$

mistake updates.

(Hint: Use the fact $\frac{\boldsymbol{\theta}_k^\top \boldsymbol{\theta}^*}{\|\boldsymbol{\theta}_k\|\|\boldsymbol{\theta}^*\|} \leq 1$. Why?)

Perceptron often terminates before this bound. A numerical bound requires a known separating vector and margin. For nonseparable data, the strict-separation assumption fails, so this guarantee does not apply.

Solution: If no update occurs, termination is immediate. For $k \geq 1$, step 2 gives $\boldsymbol{\theta}_k^\top \boldsymbol{\theta}^* \geq k\rho > 0$, hence $\boldsymbol{\theta}_k \neq 0$. Dividing the bound in step 2 by the norm bound in step 4 gives $\frac{\boldsymbol{\theta}_k^\top \boldsymbol{\theta}^*}{\|\boldsymbol{\theta}_k\|} \geq \sqrt{k}\frac{\rho}{R}$. Rearranging terms gives

$$k \leq \frac{(\boldsymbol{\theta}_k^\top \boldsymbol{\theta}^*)^2 R^2}{\|\boldsymbol{\theta}_k\|^2 \rho^2} = \frac{(\boldsymbol{\theta}_k^\top \boldsymbol{\theta}^*)^2 R^2 \|\boldsymbol{\theta}^*\|^2}{\|\boldsymbol{\theta}_k\|^2 \|\boldsymbol{\theta}^*\|^2 \rho^2} \leq \frac{R^2\|\boldsymbol{\theta}^*\|^2}{\rho^2}.$$

The last inequality follows because the squared cosine of the angle between $\boldsymbol{\theta}_k$ and $\boldsymbol{\theta}^*$ is at most 1.

Problem 5 (28 points). Perceptron and Pocket with a Coding Agent

The AI collaboration permission and workflow stated in Problem 3 apply here. Use `code_source/p5/p5.py`. Submit one additional AI collaboration case of at most one page. The two cases together must not exceed two pages.

For nonseparable data, ordinary perceptron need not terminate and its training error can fluctuate. Pocket keeps the parameter with the smallest training error seen so far. Its stored training error is therefore *nonincreasing*; there is no analogous guarantee for its test error.

Use the supplied US Post Office digit data: `train_data.mat` and `test_data.mat`. The full datasets contain 7291 training images and 2007 test images of size 16×16. Extract digits 1 and 6, assigning label $+1$ to digit 1 and -1 to digit 6. The supplied feature code computes mean pixel intensity and left–right asymmetry. Augment these with a bias coordinate:

$$\widetilde{x}_i = (1, \text{intensity}_i, \text{asymmetry}_i)^\top, \qquad \theta \in \mathbb{R}^3.$$

Predict $+1$ when $\theta^\top \widetilde{x}_i \geq 0$ and -1 otherwise. This fixes the zero-score convention; do not subtract an additional threshold from the score. Count misclassification using exactly this prediction rule.

At a mistake update, choose a misclassified *training* sample uniformly at random and set

$$\theta_{k+1} = \theta_k + y_i \widetilde{x}_i.$$

Initialize $\theta_0 = \widehat{\theta}_0 = 0$, use random seed 0, and make at most $T = 2000$ mistake updates. Stop early if no training sample is misclassified. Update the pocket only when the new training error is strictly smaller, and store a copy of the parameter. Continue the next perceptron step from θ_{k+1}, not from the pocket.

To make the comparison controlled, record the current perceptron parameter and the pocket parameter along the *same* update trajectory. Store the initial state and every subsequent state. Compute test errors from these saved states only after training has finished; test labels must not influence updates, pocket selection, or the stopping rule.

(a) **(7 points) Agent-assisted implementation.** Complete the perceptron-step and pocket-update TODOs. The supplied code handles loading, features, random sampling, trajectory collection, and plots. Explain the difference between the current parameter and the pocket parameter, and point to the code that implements each update. Record the command and seed needed to reproduce your run.

(b) **(6 points) Error curves and interpretation.** Plot both training and test errors of the current perceptron and of the pocket versus the number of mistake updates. Save the numerical trajectory as a CSV file. Explain which curve has a monotonicity guarantee and why. Describe the actual behavior of your test curves without claiming that pocket must improve test error at every step or outperform the current perceptron on every run.

(c) **(4 points) Features and decision boundaries.** Use the provided plotting code to show 500 training images from each of digits 1 and 6 in feature space, or all available images if a class contains fewer than 500. Overlay the final current-perceptron and pocket boundaries on the same axes. Label both axes and both boundaries. Explain the role of the bias coordinate. If a boundary is vertical or the normal vector is zero, handle and explain that case instead of dividing by zero.

(d) **(5 points) Audit the pocket using a small trace.** Before running the small example below, write one or two sentences predicting which error sequence should be nonincreasing and whether this property should extend to test error.

$$\widetilde{X}_{toy} = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 0 & 0 \\ 1 & 0 & 0 \\ 1 & 1 & 0 \end{bmatrix}, \qquad y_{toy} = \begin{bmatrix} 1 & -1 & 1 & -1 \end{bmatrix}^\top.$$

The first two samples have identical inputs and opposite labels, so these data are nonseparable. Use the same prediction rule and zero initialization, seed 0, and six mistake updates.

(1) Complete the history-audit TODO and save a seven-row trace (initialization plus six updates) showing the actual current and pocket weights and their training errors. Select one update from this trace and verify its error calculation and pocket decision explicitly.

(2) Check numerically that the pocket error equals the running minimum of the current training errors, including initialization:

$$E_{in}(\widehat{\theta}_k) = \min_{0 \le j \le k} E_{in}(\theta_j).$$

Explain why storing the current parameter without a copy can invalidate the intended pocket state when the current parameter is modified in place.

(3) State whether a temporary rise in pocket test error, by itself, shows that the implementation is wrong. Support your answer using the selection rule.

Do not edit a trace to force the expected property. If a check fails, retain the initial evidence and explain how you diagnosed it.

(e) **(6 points) AI collaboration case, at most one page.** Bring one concrete result from parts (b)–(d) back to your agent and ask a focused question about the implementation or the algorithm's guarantee. Include your initial prediction, tool/model, actual prompt and relevant response excerpt, numerical evidence, a relevant follow-up excerpt, and your decision with justification. For example, you might examine whether a plotted curve uses current or pocket weights, why a test curve rises, or whether saved arrays change after a later update. Use an issue relevant to your actual run. A verified correct implementation needs no artificial bug or forced revision.

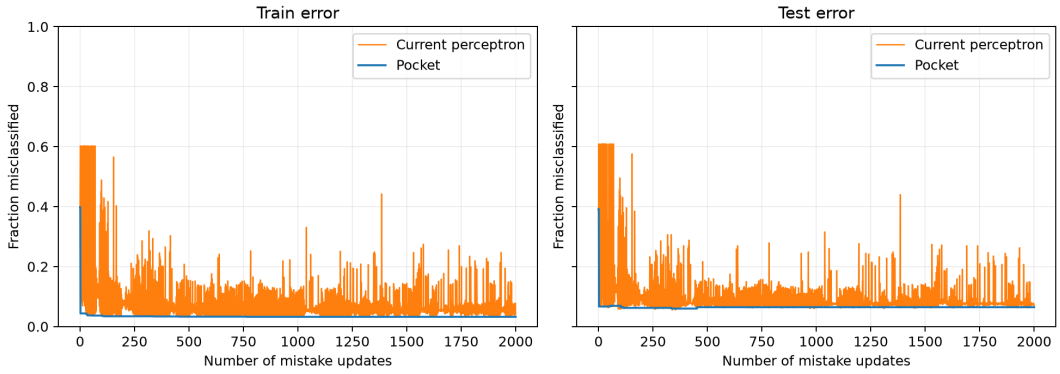
Problem 5: reference solution and marking guide

(a) The reference code is `code_solution/p5/p5.py`. It uses a three-dimensional parameter including the bias, applies the stated zero-score convention consistently, and trains one current-perceptron trajectory with a separate best-so-far pocket. The pocket is initialized using the training error of the zero parameter. A strict improvement stores a copy; ties keep the existing pocket. No test data are passed to the training function. The default run uses seed 0 and at most 2000 mistake updates. *Marking:* correct perceptron update and bias/tie handling, 3 points; correct pocket initialization, selection, and copying, 3 points; reproducible run and explanation of the two states, 1 point.

(b) The reference curves below are generated by the submitted solution code on the supplied data. After selecting digits 1 and 6, there are 1669 training and 434 test samples. Numerical errors and update counts are saved in `summary.json` and `trajectory.csv`. After 2000 updates with seed 0, the errors are:

Parameter	Training error	Test error
Current perceptron	0.040144	0.071429
Pocket	0.032355	0.064516

The current training error may rise after an update; the pocket training error cannot. Test error does not enter the pocket-selection comparison and can move in either direction. In this run the pocket test error rises at updates 54 and 453, providing concrete examples. Neither empirical monotonicity in a particular test curve nor a better final test score would establish a general guarantee.

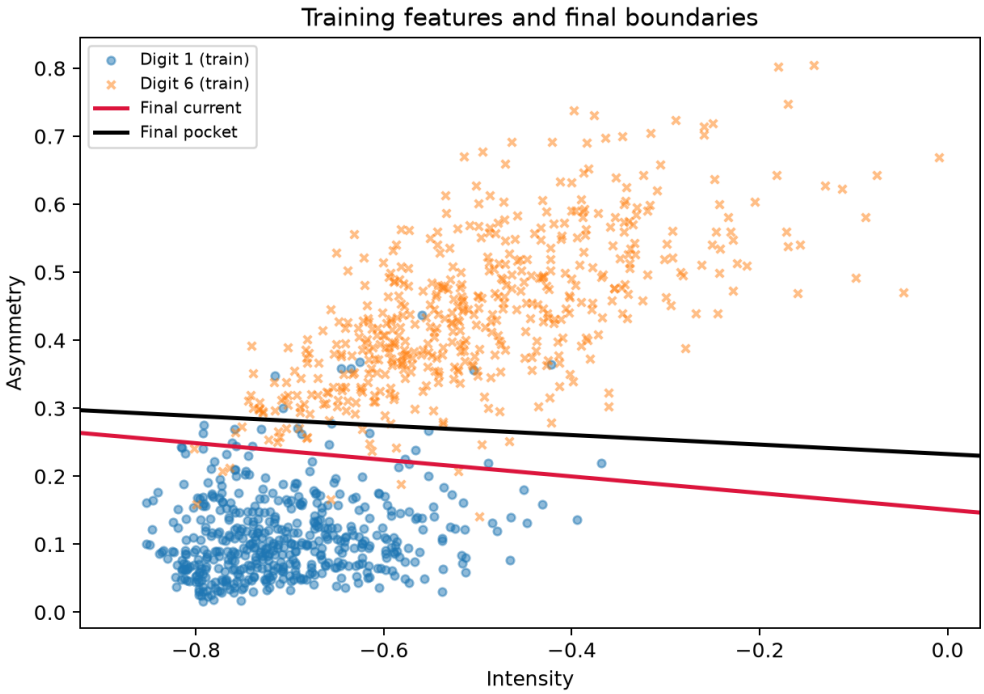


Marking: four correctly identified curves and matching data, 3 points; correct explanation of training and test behavior, 3 points.

(c) For $\theta = (\theta_0, \theta_1, \theta_2)^\top$, the boundary in feature space is

$$\theta_0 + \theta_1 \,\text{intensity} + \theta_2 \,\text{asymmetry} = 0.$$

When $\theta_2 \neq 0$, solve for asymmetry. When $\theta_2 = 0$ but $\theta_1 \neq 0$, draw the vertical line intensity $= -\theta_0/\theta_1$. A zero normal vector defines no nontrivial separating line. The bias allows a line that does not pass through the origin of feature space.



Marking: correctly sampled and labeled features, 1 point; both correct boundaries, 2 points; bias/degenerate-case explanation, 1 point.

(d) At initialization every score is zero, so every prediction is $+1$ and the toy training error is $2/4$. The supplied script exports all seven actual states in `toy_trace.csv`:

k	Current $\theta_k^\top$	Pocket $\widehat{\theta}_k^\top$	Current E_{in}	Pocket E_{in}
0	$(0,0,0)$	$(0,0,0)$	0.50	0.50
1	$(-1,-1,0)$	$(0,0,0)$	0.50	0.50
2	$(0,-1,0)$	$(0,-1,0)$	0.25	0.25
3	$(-1,-1,0)$	$(0,-1,0)$	0.50	0.25
4	$(0,-1,0)$	$(0,-1,0)$	0.25	0.25
5	$(-1,-1,0)$	$(0,-1,0)$	0.50	0.25
6	$(0,-1,0)$	$(0,-1,0)$	0.25	0.25

For example, at $k=3$ the current scores are $(-1,-1,-1,-2)$, giving two mistakes. The pocket scores remain $(0,0,0,-1)$, giving one mistake under the stated zero-score rule. The candidate therefore does not replace the pocket.

For a candidate training error e_{k+1} and a previous pocket error p_k, the new pocket error is $\min(p_k, e_{k+1})$. Induction gives the running-minimum identity including the initial state.

An assignment such as `pocket = theta` may alias a mutable array. Subsequent in-place changes to `theta` would then also change the supposed saved parameter. Copying the parameter and stored snapshots prevents this. The reference tests check both the numerical running-minimum property and independence of saved arrays.

A rise in test error does not by itself show a bug because test error is not the selection criterion. It is still appropriate to check the implementation and the plotted state, but neither an experiment nor a successful unit test replaces the argument about the selection rule. *Marking:* trace and explicit verification of a selected update, 2 points; running-minimum audit and copying explanation, 2 points; correct interpretation of test error, 1 point.

(e) There is no reference transcript to copy. Award 1 point for a specific initial prediction/checking question, 2 points for relevant genuine interaction evidence including a focused follow-up, and 3 points for a decision justified by the actual trace, code, curves, or selection rule. Credit evidence-based confirmation as well as correction. Do not award marks for conversation length or for reporting an AI mistake without verifying it.